

Node.js CI/CD Pipeline Project

Simple English project explanation: GitHub Actions + Docker + Docker Hub

Project idea

This project automatically checks and packages a Node.js web application whenever you push new code to the **main** branch of GitHub. The automation is called a CI/CD pipeline.

CI means Continuous Integration: the system installs dependencies and runs tests. **CD** means Continuous Delivery/Deployment: when tests pass, the system builds a Docker image and sends it to Docker Hub.

Main flow:

```
Developer pushes code to main
-> GitHub Actions starts
-> Install Node.js packages
-> Run test
-> Build Docker image
-> Push image to Docker Hub
```

Tools used

Tool	Simple purpose
GitHub	Stores the code and starts the automation after a push.
GitHub Actions	Runs the CI/CD steps automatically in a cloud runner.
Node.js and npm	Run the web app, install packages, and run tests.
Docker	Packs the app and its runtime into one portable image.
Docker Hub	Stores the finished Docker image so it can be pulled and run elsewhere.

What you will submit

- A GitHub repository named **nodejs-demo-app**.
- A simple Node.js web application.
- A **Dockerfile** and **.dockerignore** file.
- A GitHub Actions workflow at **.github/workflows/main.yml**.
- A Docker Hub repository containing the built image.
- A successful GitHub Actions run showing that testing, building, and pushing worked.

Step-by-step setup

1. Create the GitHub repository

Create a new GitHub repository called **nodejs-demo-app**. Clone it on your computer and open the folder in VS Code or another editor.

```
git clone https://github.com/YOUR_USERNAME/nodejs-demo-app.git
cd nodejs-demo-app
```

2. Create the Node.js app

Initialize a Node.js project and install Express. Express helps create a small web server.

```
npm init -y
npm install express
npm install --save-dev jest supertest
```

Create a file named **server.js**:

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.get("/", (req, res) => {
  res.status(200).json({
    message: "Node.js CI/CD demo app is running"
  });
});

if (require.main === module) {
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });
}

module.exports = app;
```

The app listens on port 3000. When someone opens the home page, it sends a small JSON message.

3. Add scripts in package.json

In **package.json**, make sure the scripts section has these commands:

```
"scripts": {
  "start": "node server.js",
  "test": "jest --runInBand"
}
```

Run this command after editing so that Node creates or updates **package-lock.json**:

```
npm install
```

4. Add a test

Create the folder **test** and create **test/app.test.js**. This test checks whether the home route returns status 200 and the correct message.

```
const request = require("supertest");
const app = require("../server");

describe("GET /", () => {
  it("returns a successful response", async () => {
    const response = await request(app).get("/");

    expect(response.statusCode).toBe(200);
    expect(response.body.message).toBe(
      "Node.js CI/CD demo app is running"
    );
  });
});
```

Test it before using GitHub Actions:

```
npm test
```

If the test fails, fix the application before continuing. This is important because the pipeline must not publish a broken application image.

5. Create the Dockerfile

Create a file called **Dockerfile** in the main project folder:

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm ci --omit=dev

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

What this does: it starts from a small Node.js image, copies the package files, installs only production packages, copies the app code, opens port 3000, and starts the app.

6. Add .dockerignore

Create a file named **.dockerignore**:

```
node_modules
npm-debug.log
.git
.github
Dockerfile
.dockerignore
test
coverage
.env
```

This reduces image build time and avoids copying unnecessary files into the Docker build context.

7. Test Docker on your computer

```
docker build -t nodejs-demo-app:local .
docker run -d -p 3000:3000 --name nodejs-demo-app nodejs-demo-app:local
```

Open **http://localhost:3000** in a browser. You should see the JSON message. Then stop the container:

```
docker stop nodejs-demo-app
docker rm nodejs-demo-app
```

8. Create Docker Hub credentials

- Log in to Docker Hub and create a repository called **nodejs-demo-app**.
- Create a Docker Hub personal access token with permission to push images.
- In the GitHub repository, open **Settings** → **Secrets and variables** → **Actions**.
- Create two secrets: **DOCKERHUB_USERNAME** and **DOCKERHUB_TOKEN**.

Never write the token directly in the workflow file. GitHub Secrets hides it from the repository and from most workflow logs.

9. Create the GitHub Actions workflow

Create folders **.github/workflows**. Inside that folder, create **main.yml**. Copy this full workflow:

```
name: Node.js CI/CD Pipeline

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read
```

```

env:
  IMAGE_NAME: nodejs-demo-app

jobs:
  test:
    name: Test Node.js application
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
        cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test

  build-and-push:
    name: Build and push Docker image
    runs-on: ubuntu-latest
    needs: test

    if: github.event_name == 'push' && github.ref == 'refs/heads/main'

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build and push Docker image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ${ secrets.DOCKERHUB_USERNAME }/nodejs-demo-app:latest
            ${ secrets.DOCKERHUB_USERNAME }/nodejs-demo-app:${ github.sha }

```

10. Understand the workflow

- **on: push to main:** starts the workflow when you push code to the main branch.
- **pull_request:** runs tests when a pull request targets main, but does not push an image.
- **test job:** downloads the code, sets up Node.js, installs dependencies with npm ci, and runs npm test.
- **needs: test:** makes the Docker job wait. If tests fail, Docker Hub will not receive a new image.
- **docker/login-action:** signs in to Docker Hub using GitHub Secrets.
- **docker/build-push-action:** creates the Docker image and uploads it to Docker Hub.
- **latest tag:** points to the newest successful image.
- **github.sha tag:** points to the exact Git commit, which makes the build easy to trace later.

11. Commit and push

```
git add .
git commit -m "Add Node.js Docker CI/CD pipeline"
git branch -M main
git push origin main
```

After this push, GitHub Actions starts automatically.

12. Check the result

- Open your GitHub repository and select the **Actions** tab.
- Open the latest workflow run. Both jobs should show a green check mark.
- Open Docker Hub and check that your repository has the tags **latest** and a commit SHA tag.
- Optionally pull the published image and run it locally or on a server.

```
docker pull YOUR_DOCKERHUB_USERNAME/nodejs-demo-app:latest
docker run -d -p 3000:3000 YOUR_DOCKERHUB_USERNAME/nodejs-demo-app:latest
```

Important note about deployment

This project pushes a Docker image to Docker Hub. That is often described as the deployment stage in a basic CI/CD assignment because the tested application package is published and ready to run.

To run the app on a real server, add another deployment job later. For example, the job could connect to a virtual machine, pull the new Docker image, stop the old container, and start the new version. You can also deploy the Docker image to services such as AWS ECS, Google Cloud Run, Azure Container Apps, Render, or Kubernetes.

Final checklist

- The Node.js app works locally.
- npm test passes locally.
- Docker build and Docker run work locally.
- Docker Hub repository exists.
- GitHub Secrets are added correctly.
- The .github/workflows/main.yml file is committed.
- You push to main and the GitHub Actions workflow passes.
- The image appears in Docker Hub with latest and commit-SHA tags.